

Model Assessment (Evaluation/Validation) and Model Selection (part I)

Alessio Micheli

micheli@di.unipi.it

2025



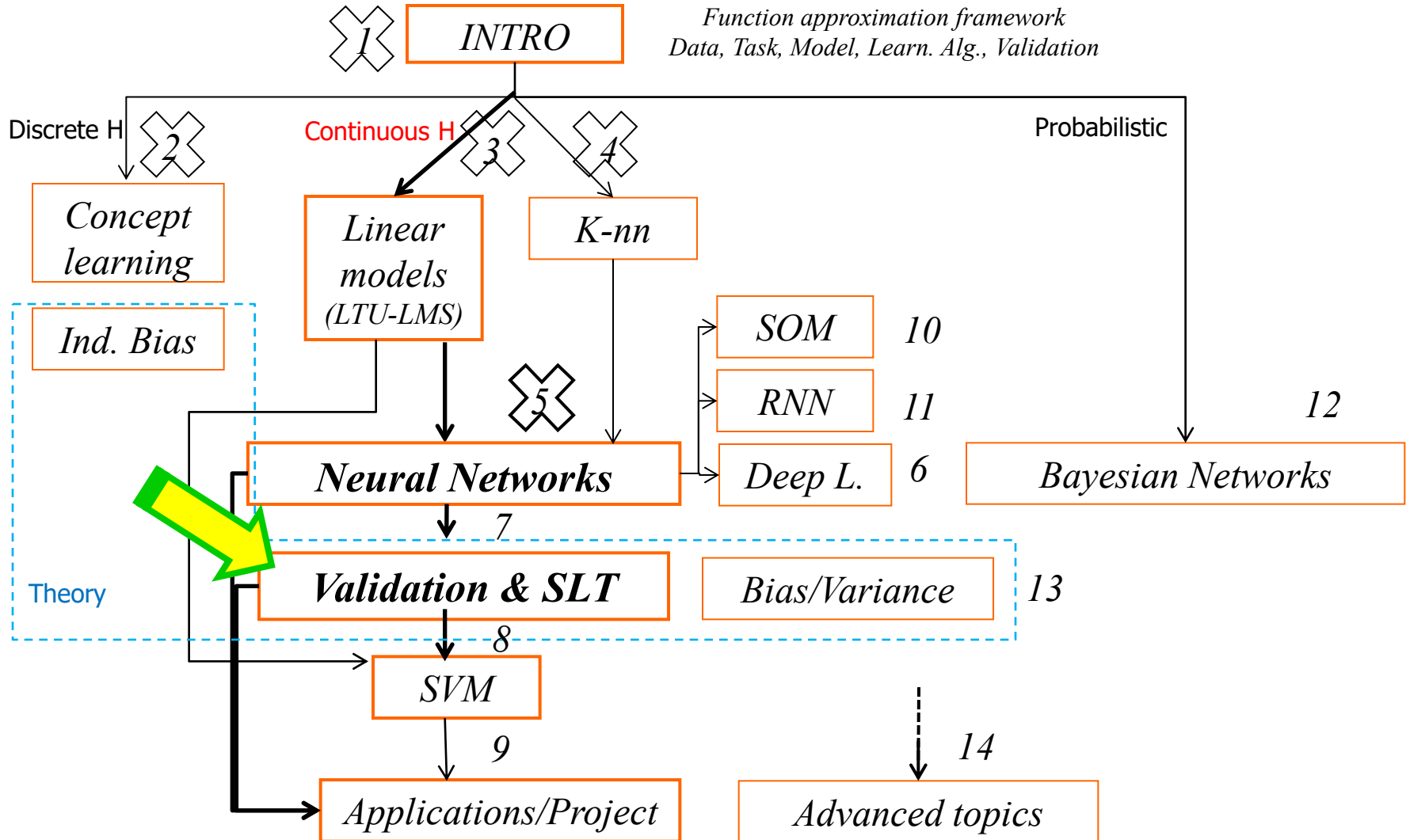
Dipartimento di Informatica
Università di Pisa - Italy

ML Course structure

Where we go



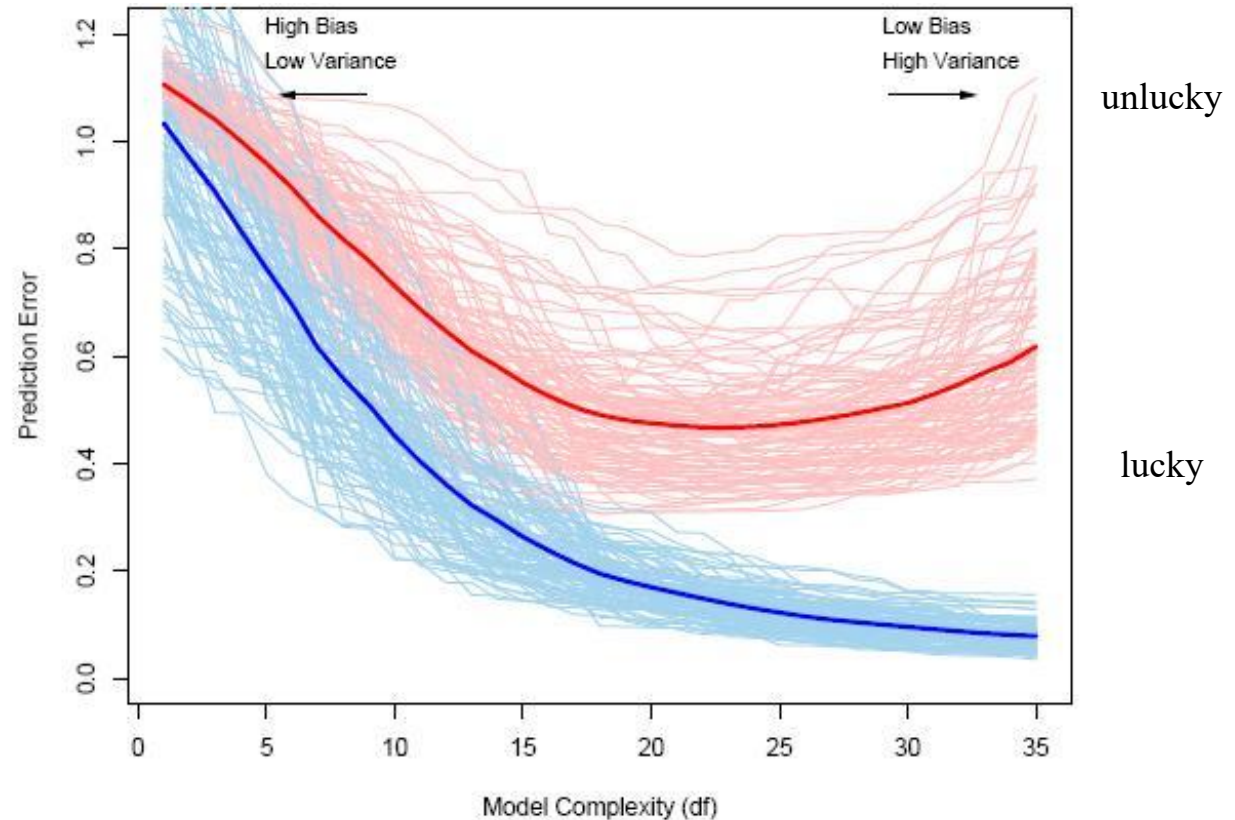
Dip. Informatica
University of Pisa



A premise: Bias-Variance

- Bias-Variance decomposition provides an useful framework to understand the validation issue, showing how the estimation of a model performance *is difficult*
 - Considering also the role of different training set realizations
 - Showing again the need of a trade-off between fitting capability (bias) and model flexibility (variance) in different way
- Anyway we will discuss it later by a specify lecture
- Let me introduce just this plot, to recall the need to optimize a model in the right way.

Under & Overfitting in a Bias-Variance plot*



Complex model : lower
bias, higher variance →
possible overfitting

FIGURE 7.1. Behavior of test sample and training sample error as the model complexity is varied. The light blue curves show the training error $\overline{\text{err}}$, while the light red curves show the conditional test error $\text{Err}_{\mathcal{T}}$ for 100 training sets of size 50 each, as the model complexity is increased. The solid curves show the expected test error Err and the expected training error $E[\overline{\text{err}}]$.

Motivations

- Recall that we are looking for the best solution (minimal prediction – test error) searching a **balancing** between **fitting (accuracy on training data)** and **model complexity**
 - Training set **is not** a good estimate of test error
 - Initially ok but you have too many bias (underfitting, high TR error) than too high variance (low TR error but overfitting)
- Assuming a tuning hyper-parameters θ (implicit or explicit) that varies the model complexity, we wish to find the value of θ that minimizes test error
 - Look for methods for estimating the expected error for a model (or each model of a class of models or even of a set of models*)

Approximating estimation step (intro: a guide)



Dip. Informatica
University of Pisa

We can approximate the estimation

- Analytically:
 - AIC, BIC Akaike/Bayesian Information Criterion
 - limited to linear models (in par.)
 - MDL (Minimum Description Length)
 - SRM: **Structural Risk Minimization & VC-dimension** →
 - In lecture INTRO + a next lecture on the VC-DIM

In practice, often we can approximate the estimation on the data set:

- By resampling: direct estimation of extra-sample error via
 - cross-validation (hold-out, K-fold CV, ...)
 - bootstrap

Validation: Two aims

Recall the slides in the intro (on the validation)! **THE MOST IMPORTANT SLIDE** of the course

- After models training on the training set
- **Model selection:** estimating the performance (*generalization error*) of different learning models in order to choose the best one (to generalize)
 - this includes search the best *hyper-parameters* of your model (e.g. polynomial order, #units in a NN, lambda, eta,).

It returns a model

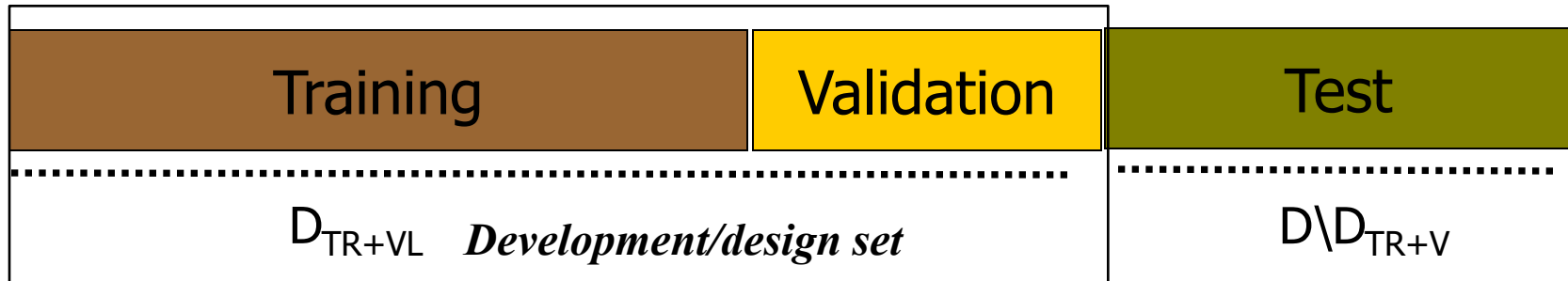
- **Model assessment:** having chosen a final model (or a class of models), estimating/evaluating its prediction error/ risk (*generalization error*) on new test data (measure of the quality of the ultimately chosen model)

It returns an estimation value

Gold rule: Keep separation between goals and use separate data sets in each phase

Hold out

- If data set size is sufficient: e.g. 50% TR, 25% VL, 25% TS **disjoint sets**



- TR**: *Training set* is used to fit (to minimize R_{emp} in SLT) [**training**]^L
- VL**: *Validation set* (or *selection set*) is used to select the best model (among different models and/or hyper-parameters configurations) [**model selection**]
- TR+VL sometimes are jointly called development/design set, i.e. used to build the final model (training different models and selecting the best final model)
- TS**: *Test set* is used for estimation of generalization error (of the final model) (to estimate R in SLT) [**model assessment**]

Notes:

1) the estimation made for model selection (on VL set) [is for model selection purpose], it is not a good estimation for the assessment phase/risk test.

2) **Test set results** cannot be used for model selection (or call it validation set)

Test or model selection?

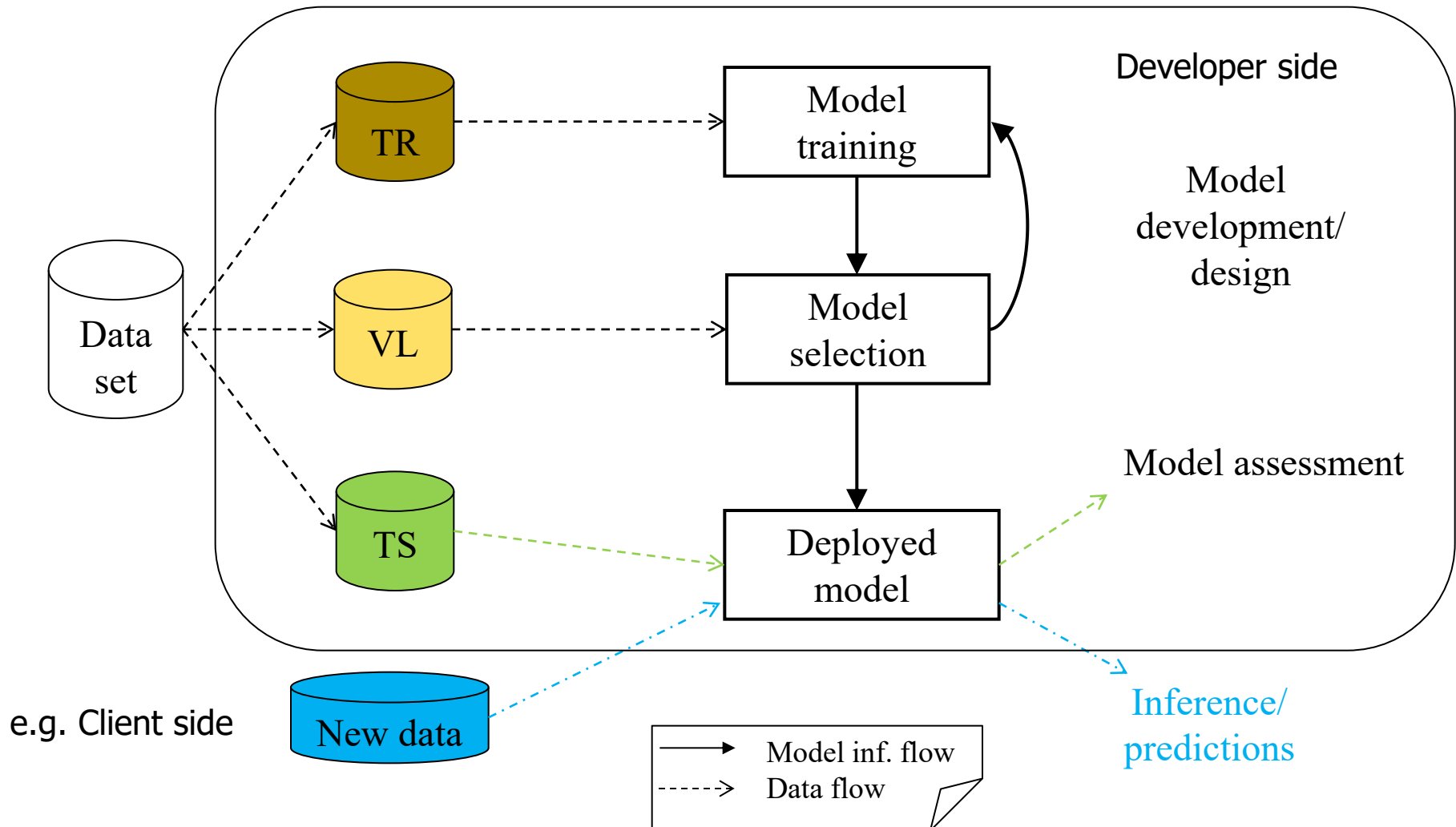
- What if test set is used in a (repeated) design cycle?
 - We are making *model selection* and not reliable *assessment* (estimation of expected generalization error)
 - and *we wouldn't be able to do that on future examples*.
 - **Blind test set** concept (e.g. for ML competitions)
 - Image an exam exercise: if you see the solutions, it is not a test!
 - In that case, used test set error provides an overoptimistic evaluation of the true test error (→ we will see how easy is to obtain very high classification accuracy over random task even using the test set only implicitly)

Gold rule:

Keep separation between goals and use separate sets in each phase
(TR for training, VL for model selection, TS for risk estimation)



TR/VL/TS by a simple schema



Counterexample

(relevance of a correct assessment)

- 20-30 examples, 1000 random value input variables,
- **Random** target 0/1
- We select 1 model with 1 input var/feature that guess (for chance) 99% (or 100%) on any successive TR and VL and TS set splitting

? Perfect result (a model with accuracy 99%)? What is wrong?

99%/100% is not good estimation of the test error (the right one is 50%)

1) Estimation of error on TR and VL is NOT good risk estimation.

2) Using the entire dataset for feature/model selection prejudice the estimation (biased estimation, called *subset selection bias*)

- Test set was implicitly used at the beginning (*).
- Test set must be separated in advance, before making any model selection (even of the Feature selection) !

On external test the result is 50% (random coin result)!!!!

We are discussing the correctness of the estimation, not the possibility to solve the task !

K-fold CV and other techniques does not solve the issue
[but can confuse you ! ;-)]

The table for the Counterexample



Dip. Informatica
University of Pisa

Pattern

Input variable value

	1	2	...	26	27	28	...	1000	Target
1	...	...	...	1	1	1	...	...	1
2				0	0	0			0
3				1	1	1			1
4				0	0	0			0
...				0	0	0			0
...				1	1	1			1
...				0	0	0			0
20	...	...	...	1	1	1	...	...	1
TS1				1	0	0			1
TS2				0	1	0			0
TS3				1	1	1			1
Accuracy				100%	33%	66%			

Grid search (model selection phase)



Dip. Informatica
University of Pisa

- *Hyper-parameters*. i.e. parameters that are not directly learnt within estimators, can be set by searching a *hyper-parameters* space (of course by model selection on a *validation set*). Search **best** hyper-parameter values.

- **Exhaustive Grid Search**

exhaustively generates candidates from a grid of parameter values,

TRIVIAL EXAMPLE:

Hyper-param.	Lambda 0.1	Lambda 0.01	Lambda 0.001
1 unit	Res1	Res4	Res7
10 units	Res2	Res5	Res8
100 units	Res3	Res6	Res9

Example: The best one is **Res3** →
(Units=100, lambda=0.1) is the winner

AUTOMATIZE IT!!!
Parallelization is easy
(independence of trials)

- Alternatives: Randomized Parameter Optimization (*) → see later
- And many others (also in current ML libraries) → see later

Grid search -2

- The cost of search can be high: Cartesian product between the sets of values for each hyper-parameter
- $\#(\text{values in the range})^{\# \text{hyperparameters}}$:
 - e.g. before $3 \times 3 = 3^2 = 9$
 - What with 5-6 hyperpar., each with 10 possible different values?

Hyper-param.	Lambda 0.1	Lambda 0.01	Lambda 0.001
1 unit	Res1	Res4	Res7
10 units	Res2	Res5	Res8
100 units	Res3	Res6	Res9

- Can be useful to fix some hyper-parameters values in a preliminary experimental phase (since they show to be not relevant) [but take care, see FAQ later on seq. selection error]

Two (or more) levels of **nested grid search**:



- Apply a first *coarse grid* search to the hyperparameters with a table on the combinations of **all the possible** (e.g. growing exponential) values too find good intervals (regions)
- Then finer grid-searches can be performed (over smaller and smaller intervals and with selected hyper-parameters) still using **all** the significant hyper-par.

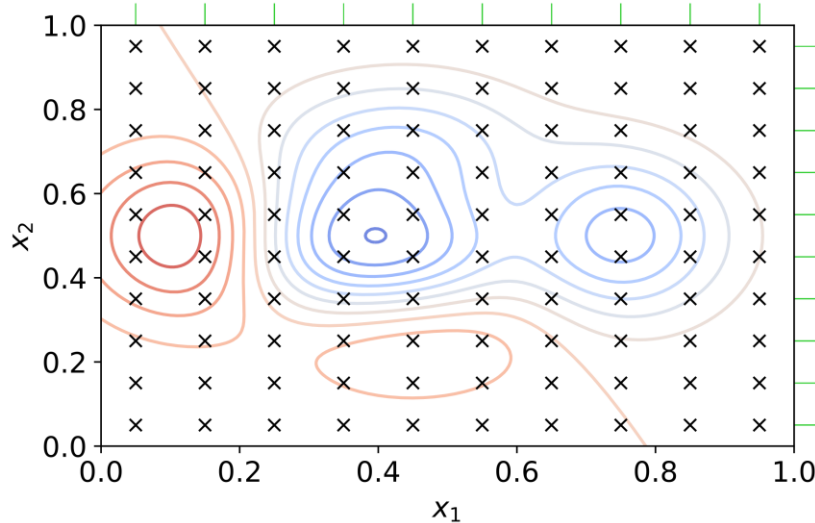
Alternatives to grid search



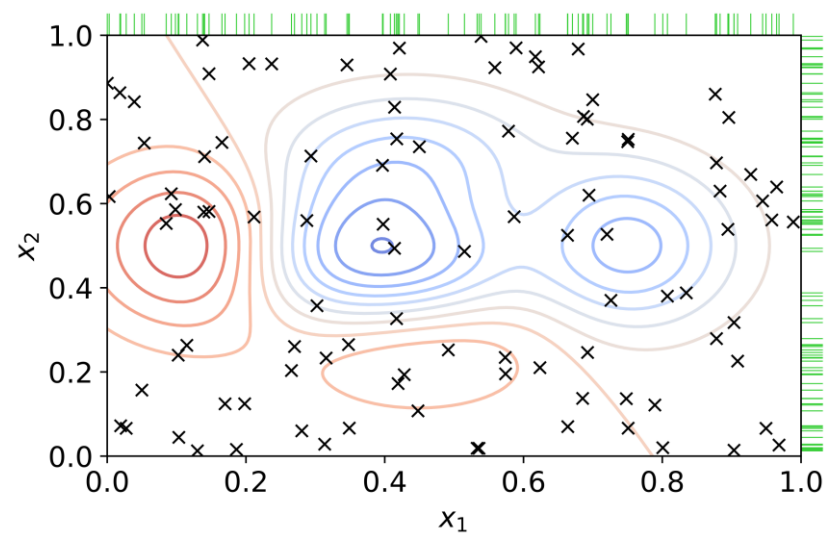
Dip. Informatica
University of Pisa

- Grid search is very useful to observe the behavior of the model changing the hyper-parameters values (to explore the hyper-parameter space → very important for your understanding)
 - But it is *computationally demanding* for large spaces and does not scale well with the increase in number of hyperparameters
- Alternatives exist, trying:
 - to reduce the cost: e.g. *random search*, which avoid to resample the same values of influent hyperparameters (in case some other are not-influent), and allows to *fix the budget* of samples independently from the # of hyperpar. See Bergstra, Y. Bengio, (2012). "Random Search for Hyper-Parameter Optimization" J. Machine Learning Research 13: 281–305.
 - to automatize the search: e.g. *Bayesian approaches, evolutionary approaches, configuration evaluation methods* (e.g. Hyperband*), et al.
- Current libraries, as Keras tuner for NN and Scikit-Learn, include various alternatives (toward *AutoML*) → anyway, don't try them uncritically, compare with the grid search that teaches you more for the first approach to ML

Grid versus Random search



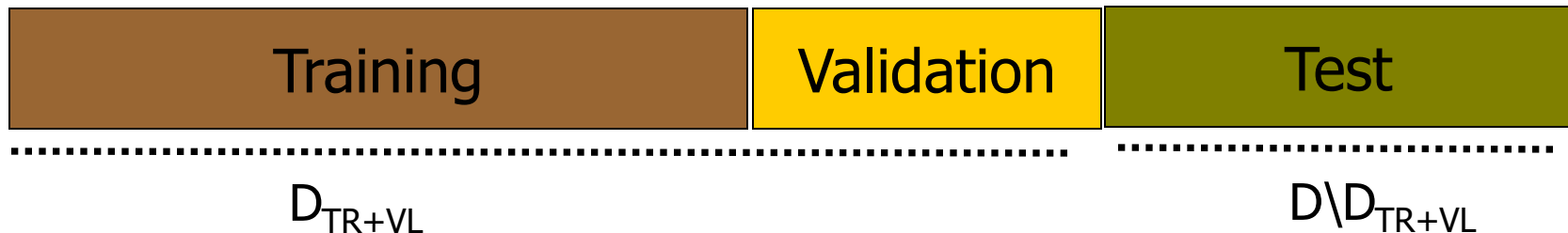
Grid search for 2 hyperparameters:
a total of 100 (10x10) different combinations are evaluated and compared. Blue contours indicate regions with strong results, whereas red ones show regions with poor results.
ONLY 10 samples for each axis!



Random search for 2 hyperparameters:
100 different random choices are evaluated. The green bars show that more individual values for each hyperparameter are considered compared to a grid search.
MANY samples for each axis

Hold out - rethinking

- If data set size is sufficient: [e.g.](#) 50% TR, 25% VL, 25% TS



- How much data is enough?
 - It depends on model complexity, signal-to-noise ratio of the underlying function (e.g. Few for linear model, linear separable task, no noise)
- How to deal with data that are insufficient to make 3 significant/large size splits?
- Can we avoid to be sensible to the particular partitioning of examples?

K-fold CV can help!

Notes and Terminologies

- Generically we have used the term "*cross validation*" for both:
 - An approach to ***model selection*** based on direct estimation of prediction error **by resampling** (instead of analytical)
 - Choose hyperparameters estimating errors
 - More specifically for implementations of estimation methods with resampling specifying how to divide data for both *model selection and assessment*
 - How to make partitions of the data set : e.g. by **K-fold CV**

K-fold CV recall

K-fold Cross-Validation



- Split the data set D into k mutually exclusive subsets $D_1, D_2, \dots, D_K$
- Train the learning algorithm on $D \setminus D_i$ and test it on D_i
- *It uses all the data for training and testing*
- You don't get an unique model (see next lect.) but
- You get also a **variance** (standard deviation) over different folders for your **<class of models/alg.>**

It can be used both for the validation set or for the test set

Issues:

- How many folds? 3-fold, 5-fold, 10-fold, ..., 1-leave-out
- Often computationally very expensive
- Combinable with validation set, double-K-fold CV,

Fold-1	Res on D_1
Fold-2	Res on D_2
Fold-3	Res on D_3
...	...
TOTAL	MEAN +/- SD

An **example** of model selection and assessment with **K-fold CV**



Dip. Informatica
University of Pisa

- Split data in **TR** and **Test** set (here simple hold-out or a K-fold CV)
- **[Model selection]** Use K-fold CV (internal) over **TR** set, obtaining new **TR** e **VL** set in each folder, to find best hyper-parameters of your model (e.g. polynomial order, λ of ridge regression, #units in a NN, ...):
Grid-search with many possible values of the hyper-parameters
 - i.e. for example a k-fold-CV for ($\lambda = 0.1$, #units=20, ...), a k-fold-CV for ($\lambda = 0.01$ #units=20), ... i.e. k-fold-CV for each cell of the grid matrix, ... and then take the best configuration (λ , #units, ...) (comparing the mean error computed over the validation sets obtained by the all the folds of each k-fold CV, i.e. the result on the “diagonal”)
- Train on the *whole* (original) **TR** set the final model
- **[Model assessment]** Evaluate it on the external **Test** set
 - Question: Can we retrain the model again with all the data?

We will see more combinations in the next lecture!

Before...

- Before enter in general method for model selection/assessment we face (in brief) some particular cases



Lucky/Unlucky sampling

- Can we avoid to be sensible to the particular partitioning of examples? (bias of the particular sample)
- Stratification procedure:
 - *Stratification* is the process of grouping members of the population into relatively homogeneous subgroups before sampling.
- Classification: for each (random) partition (TR and TS in hold-out/CV) each class is represented in approximately the same proportions as in the full data set
- Folder composition: check if order in data related to specific meaning
- Repeated hold-out method or CV: repeat splitting with different random sampling
 - E.g. repeat the CV 10 times !
 - Average the results to yield an overall estimation
 - Useful especially for comparing different models !

Very few data (informal)



Dip. Informatica
University of Pisa

- With very few data: difficult to say whether a sample is representative or not
- Stratification
- Avoid (or consider in evaluation):
 - Missing classes or features in training data
 - Special classes of data not sampled in TR or TS (again stratification)
 - Prior known outliers can affect the mean test results
 - On the opposite: selection of only “easy” cases
- Also: blind test set can be misleading if it is
 - From a different distribution
 - Measured with different scale, different tolerance, etc.
 - Uncleaned, unpreprocessed, ...
 - Extrapolation (out of range data)

Error Functions for Evaluation

- Measuring error (addenda to slides in the intro)
- Classification: see first lessons:
 - **Accuracy** (correct classification rate/error rate often in %), confusion matrix (specificity, sensitivity, ROC curve)
- Regression: residue $r_i = (y_i - o_i)$ $y = target$
 - **MS error** ($mean_i [r^2]$) $\rightarrow$ **RMS** (root mean square) or S
 - **Mean absolute error** (mean of the absolute error $|r|$)
 - **Max absolute error** ($max_i [|r_i|]$)
 - **R** (correlation coefficient/index) (R^2 coeff. of determination):
E.g. $R = \sqrt{1 - (S^2/S_y^2)}$ where $S_y^2 = mean_i [(y_i - mean_i [y])^2]$ (variance of y)
 - degree of linear dependence between the variables y e o in $[0-1]$, where 1 is the best
 - **Outputs versus targets plots**
 - Various **statistical tests** and statistical significance analysis

Other approaches



Dip. Informatica
University of Pisa

- Bootstrap: random resampling with replacement and repeat it to form different subsets (vl or ts)

REPETITA: 2 gold rules

- TR result is not a good estimation of VL and TS result
- VL result is not a good estimation of TS result

Hence:

- Do **not** use validation set results for test estimation [*risk estimation/model assessment*]
- Do **not** use test set results for *model selection* (in any form*!!!) (or call it validation set)

Or even **1 rule**:

Keep separation between goals and use separate sets in each phase



In the following

- Correct use of cross-validation methods for model selection and/or estimation of risk (assessment)
- Simple concepts with a rich formal notations, can help to avoid ambiguities:
- Very useful guide. It is not intended as a recipe book but help you to rationalize, to understand in a systematic form issues of a rigorous approach.
- Then you have to be *reasonable (when you are aware)*
- In any case use them if you have uncertainties, it is better than to use *fantasy!*

Exercise (in the slide of the next lecture):

relate wrong cases 1 and 2 of slide "counterexample" to misuses of such schemas!

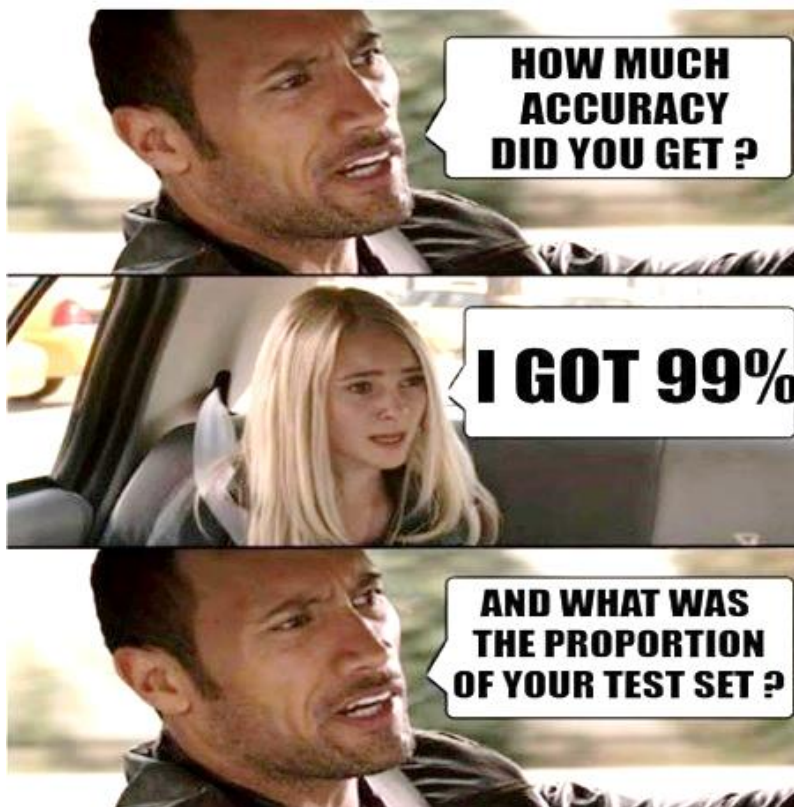
Bibliography

- Hastie, Tibshirani, Friedman: Chapter 7
- Duda, Hart, Stork: 9.4, 9.6
- Cherkassky, Mulier: 3.4.2
- Haykin: e.g. 4.14 2nd edition - 4.13 3rd edition
- Mitchell: e.g. 4.6.5, 5.1, 5.6
- Note that often the space in such books is small with respect to the relevance of the topic!!!

For fun

- Can I have just a look to the test set?

See <https://youtu.be/XvOsh15hLIs>



By Sepe-Dukic past students



* It can hold also for the validation set used as test set ;-)